

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Q1. Generate a list of 100 integers containing values between 90 to 130 and store it in the variable `int_list`. After generating the list, find the following:

(i) Write a Python function to calculate the mean of a given list of numbers. Create a function to find the median of a list of numbers.

(ii) Develop a program to compute the mode of a list of integers.

(iii) Implement a function to calculate the weighted mean of a list of values and their corresponding weights.

(iv) Write a Python function to find the geometric mean of a list of positive numbers.

(v) Create a program to calculate the harmonic mean of a list of values.

(vi) Build a function to determine the midrange of a list of numbers (average of the minimum and maximum).

(vii) Implement a Python program to find the trimmed mean of a list, excluding a certain percentage of outliers.

```
In [2]: int_list=np.random.randint(90,130,100)
int_list
```

```
Out[2]: array([106, 102, 123, 126, 124,  99, 123, 106,  95,  97, 101, 106, 116,
        123, 110,  99, 110, 127, 112, 113,  96, 108, 120, 119, 120, 127,
        123,  94, 128,  95,  94, 103, 102, 127, 126, 101,  91,  97, 119,
        120,  96,  93,  96, 112,  91,  95,  93, 111,  93, 106, 110, 128,
        108,  97, 124, 123, 108, 118, 125,  97, 110, 121, 109, 111, 123,
        127, 108, 102, 119, 125,  91,  94, 109, 105, 127, 116, 122, 122,
        110, 104,  99, 106,  93, 111, 107, 122,  91, 119, 102, 114, 111,
        98,  90,  99, 118, 124, 110, 123, 105, 117])
```

(i) Write a Python function to calculate the mean of a given list of numbers. Create a function to find the median of a list of numbers.

```
In [3]: mean_int_list=np.mean(int_list)
median_int_list=np.median(int_list)
print("Mean:",mean_int_list)
print("Median:",median_int_list)
```

```
Mean: 109.66
Median: 110.0
```

(ii) Develop a program to compute the mode of a list of integers.

```
In [4]: import statistics as stats
mode_int_list=stats.mode(int_list)
print("Mode:",mode_int_list)
```

(iii) Implement a function to calculate the weighted mean of a list of values and their corresponding weights.

```
In [5]: def calculate_weighted_mean(values, weights):
        if len(values) != len(weights):
            return "Error: The number of values and weights should be the same."
        weighted_sum = 0
        total_weight = 0
        for i in range(len(values)):
            weighted_sum += values[i] * weights[i]
            total_weight += weights[i]
        weighted_mean = weighted_sum / total_weight
        return weighted_mean

values = int_list
weights = int_list
result = calculate_weighted_mean(values, weights)
print("The weighted mean is:", round(result,2))
```

The weighted mean is: 110.87

```
In [6]: #Using numpy:
weighted_mean=np.average(int_list,weights=int_list)
print("The weighted mean is:", round(weighted_mean,2))
```

The weighted mean is: 110.87

```
In [7]: v = int_list
w = int_list
def weighted_mean(v,w):
    return sum(x*y for x,y in zip(v,w))/sum(w)

print(round(weighted_mean(v,w),2))
```

110.87

(iv) Write a Python function to find the geometric mean of a list of positive numbers.

```
In [8]: def calculate_geometric_mean(values):
        return np.exp(np.mean(np.log(values)))

values = int_list
result = calculate_geometric_mean(values)
print("The geometric mean is:", result)
```

The geometric mean is: 109.04969524412776

```
In [9]: # or use one line code
geo_mean = np.exp(np.mean(np.log(int_list)))
geo_mean
```

Out[9]: 109.04969524412776

(v) Create a program to calculate the harmonic mean of a list of values.

```
In [10]: def calculate_harmonic_mean(values):
        harmonic_mean = 1/np.mean(1 / np.array(values))
        return harmonic_mean

values = int_list
result = calculate_harmonic_mean(values)
print("The harmonic mean is:", result)
```

The harmonic mean is: 108.43686474429205

(vi) Build a function to determine the midrange of a list of numbers (average of the minimum and maximum).

```
In [11]: def Calculate_midrange(l):
          max_ = max(l)
          min_ = min(l)
          midrange = (max_ + min_) / 2
          return midrange
          Calculate_midrange(int_list)
```

Out[11]: 109.0

(vii) Implement a Python program to find the trimmed mean of a list, excluding a certain percentage of outliers.

```
In [12]: def calculate_trimmed_mean(num_list, percentage):
          sorted_list = sorted(num_list)
          exclude_count = round((percentage / 100) * len(sorted_list))
          if exclude_count == 0:
              return sum(sorted_list) / len(sorted_list) #no trimming

          trimmed_list = sorted_list[exclude_count:-exclude_count]
          return sum(trimmed_list) / len(trimmed_list)
          calculate_trimmed_mean(int_list, 10)
```

Out[12]: 109.725

```
In [13]: # Or using numpy
          def trimmed_mean(values, trim_percent):
              """
              Calculate the trimmed mean by removing a percentage of smallest and largest values.
              """
              if not 0 <= trim_percent < 50:
                  raise ValueError("trim_percent must be between 0 and 50")

              arr = np.sort(np.array(values))
              n = len(arr)
              k = int(n * trim_percent / 100)

              trimmed = arr[k:n-k] # exclude outliers from both ends
              return np.mean(trimmed)

          result = trimmed_mean(int_list, 10) # trim 10% from each side
          print("Trimmed Mean:", result)
```

Trimmed Mean: 109.725

Q2. Generate a list of 500 integers containing values between 200 to 300 and store it in the variable "int_list2". After generating the list, find the following:

(i) Compare the given list of visualization for the given data:

1. Frequency & Gaussian distribution
2. Frequency smoothened KDE plot
3. Gaussian distribution & smoothened KDE plot

(ii) Write a Python function to calculate the range of a given list of numbers.

(iii) Create a program to find the variance and standard deviation of a list of numbers.

(iv) Implement a function to compute the interquartile range (IQR) of a list of values.

(v) Build a program to calculate the coefficient of variation for a dataset.

(vi) Write a Python function to find the mean absolute deviation (MAD) of a list of numbers.

(vii) Create a program to calculate the quartile deviation of a list of values.

(viii) Implement a function to find the range-based coefficient of dispersion for a dataset

```
In [14]: int_list2=np.random.randint(200,300,500)
int_list2
```

```
Out[14]: array([242, 247, 263, 209, 245, 287, 223, 257, 216, 230, 228, 235, 210,
 209, 297, 255, 250, 214, 256, 288, 230, 222, 215, 242, 211, 271,
 203, 204, 287, 278, 211, 262, 295, 210, 219, 254, 292, 264, 266,
 277, 288, 254, 273, 298, 235, 245, 261, 202, 249, 281, 299, 237,
 292, 299, 244, 251, 241, 254, 257, 235, 275, 225, 295, 269, 240,
 275, 251, 264, 260, 208, 279, 282, 251, 229, 253, 206, 229, 217,
 213, 253, 262, 294, 262, 237, 256, 260, 280, 283, 241, 261, 213,
 214, 251, 221, 235, 246, 245, 273, 201, 228, 205, 215, 249, 299,
 299, 239, 253, 289, 228, 216, 247, 205, 277, 241, 217, 235, 271,
 240, 266, 217, 288, 203, 284, 203, 287, 248, 206, 274, 266, 205,
 285, 261, 262, 234, 266, 238, 217, 246, 208, 233, 271, 231, 220,
 226, 219, 206, 270, 243, 242, 209, 259, 283, 287, 281, 246, 222,
 256, 293, 284, 205, 210, 287, 203, 276, 262, 273, 218, 287, 235,
 263, 240, 274, 221, 212, 247, 276, 272, 263, 269, 213, 250, 256,
 285, 228, 200, 289, 258, 260, 231, 249, 260, 277, 235, 218, 241,
 224, 204, 281, 219, 247, 239, 208, 283, 287, 257, 257, 252, 295,
 258, 234, 283, 220, 251, 250, 257, 245, 220, 207, 295, 297, 244,
 265, 204, 243, 209, 252, 265, 284, 243, 274, 242, 225, 200, 263,
 262, 225, 243, 264, 233, 220, 258, 238, 294, 222, 232, 267, 267,
 294, 260, 243, 245, 210, 221, 298, 251, 227, 228, 260, 249, 239,
 254, 201, 283, 296, 223, 237, 275, 259, 246, 292, 203, 293, 283,
 219, 274, 244, 232, 212, 261, 200, 227, 254, 260, 250, 265, 219,
 214, 270, 248, 279, 227, 265, 273, 205, 293, 240, 255, 243, 224,
 208, 253, 280, 284, 211, 201, 235, 272, 284, 293, 221, 243, 298,
 289, 227, 246, 253, 269, 255, 292, 277, 298, 289, 282, 221, 261,
 279, 219, 252, 296, 233, 262, 268, 277, 295, 256, 237, 241, 295,
 228, 267, 240, 267, 273, 223, 259, 211, 277, 274, 234, 208, 239,
 267, 260, 282, 216, 223, 258, 268, 289, 267, 262, 231, 263, 216,
 250, 226, 246, 293, 279, 256, 283, 271, 217, 219, 282, 284, 227,
 270, 276, 288, 271, 225, 202, 289, 254, 254, 261, 205, 238, 292,
 250, 248, 229, 260, 235, 213, 279, 202, 264, 293, 219, 269, 267,
 290, 236, 281, 250, 273, 243, 234, 218, 271, 207, 261, 232, 217,
 251, 295, 208, 241, 205, 257, 270, 262, 283, 290, 240, 234, 236,
 278, 225, 232, 291, 247, 226, 273, 282, 269, 283, 204, 242, 243,
 295, 234, 224, 260, 268, 222, 271, 296, 299, 245, 249, 207, 273,
 247, 234, 277, 236, 263, 243, 297, 203, 200, 296, 236, 240, 283,
 247, 221, 232, 225, 216, 240, 275, 297, 207, 297, 265, 264, 243,
 296, 249, 267, 281, 222, 206, 219, 238, 270, 297, 296, 202, 244,
 216, 230, 209, 212, 295, 222])
```

(i).Compare the given list of visualization for the given data:

1. Frequency & Gaussian distribution: This visualization shows the frequency of data points along with a Gaussian distribution curve. It helps in understanding the distribution of the data and how closely it aligns with a normal distribution.

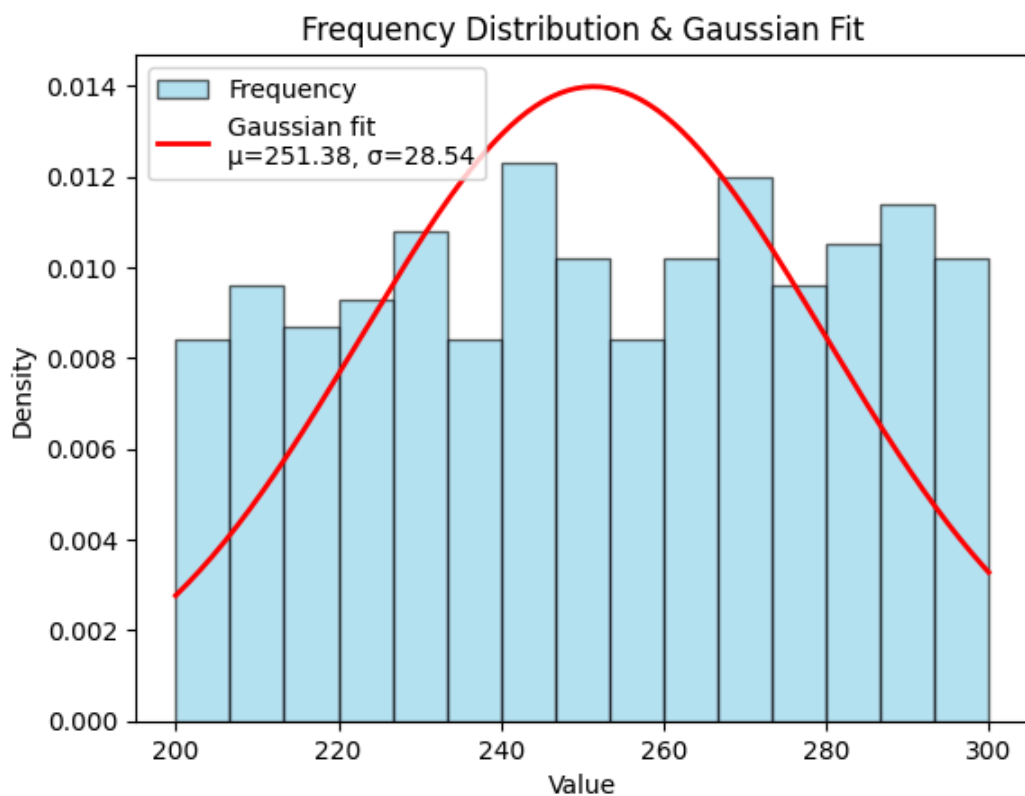
```
In [15]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Step 1: Generate the List
int_list2 = np.random.randint(200, 301, 500)

# Step 2: Plot frequency (histogram)
plt.hist(int_list2, bins=15, density=True, alpha=0.6, color='skyblue', edgecolor='black', 1

# Step 3: Fit a normal distribution & plot Gaussian curve
mu, sigma = np.mean(int_list2), np.std(int_list2)
x = np.linspace(min(int_list2), max(int_list2), 200)
pdf = norm.pdf(x, mu, sigma)
plt.plot(x, pdf, 'r', linewidth=2, label=f'Gaussian fit\nμ={mu:.2f}, σ={sigma:.2f}')

# Step 4: Labels & Legend
plt.title("Frequency Distribution & Gaussian Fit")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.show()
```

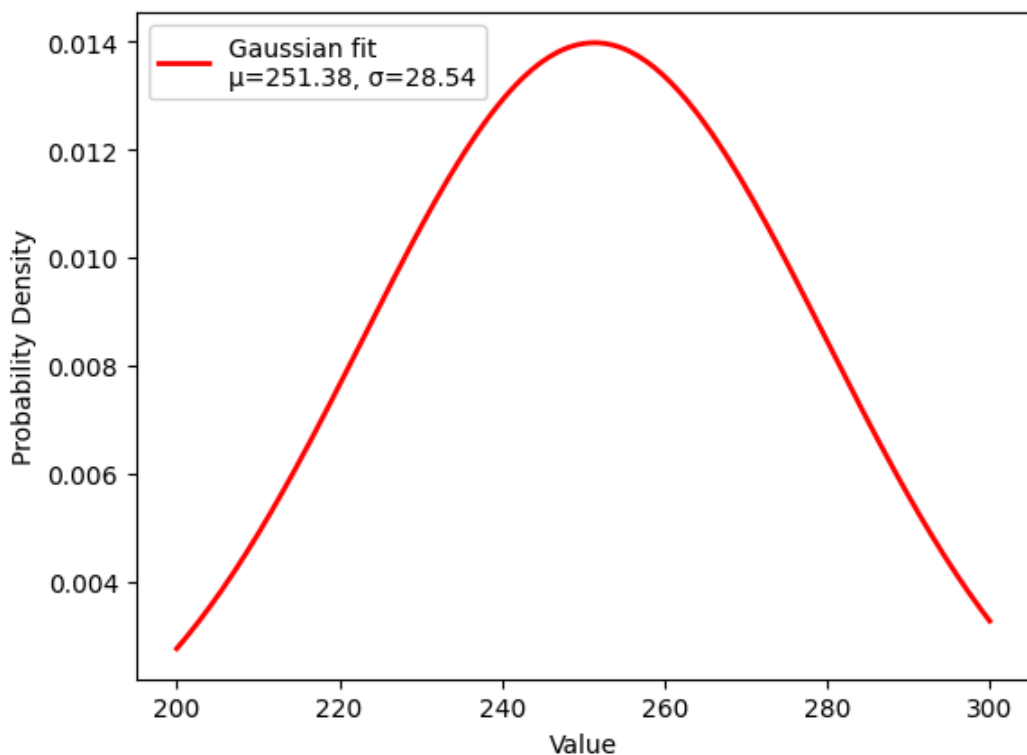
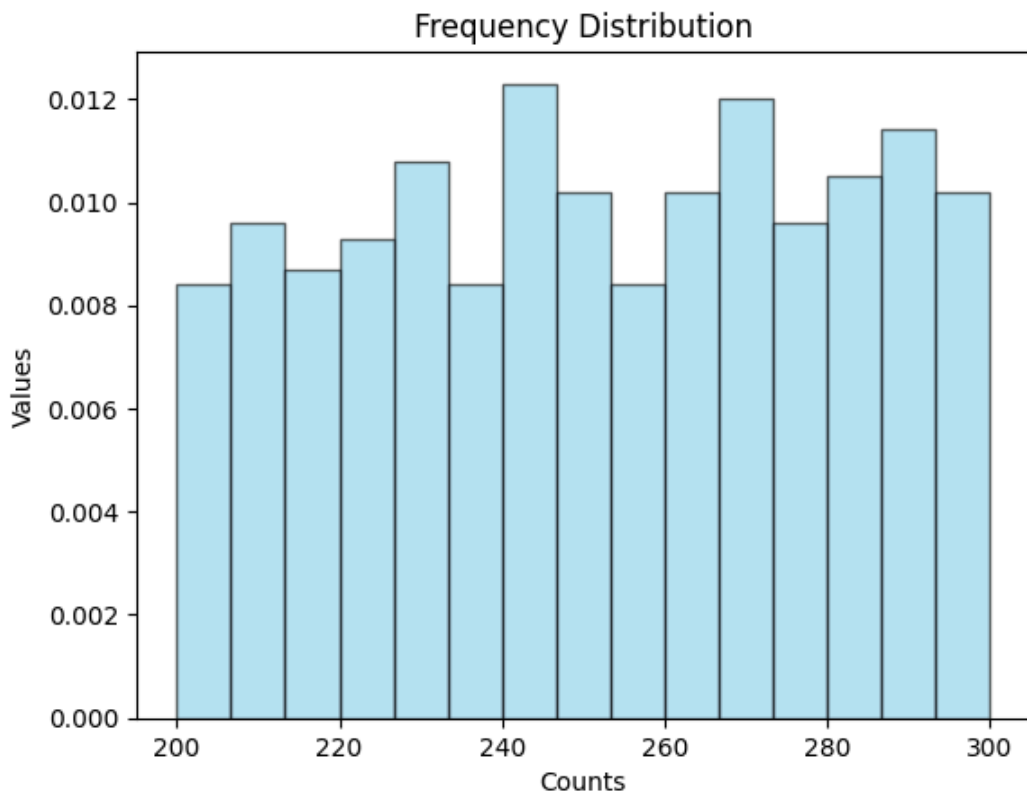


```
In [16]: import matplotlib.pyplot as plt

#Frequency distribution
plt.hist(int_list2, bins=15, density=True, alpha=0.6, color='skyblue', edgecolor='black', 1
plt.ylabel("Values")
plt.xlabel("Counts")
plt.title("Frequency Distribution")
plt.show()

#Gaussian curve
mu, sigma = np.mean(int_list2), np.std(int_list2)
x = np.linspace(min(int_list2), max(int_list2), 200)
y = norm.pdf(x, mu, sigma)
plt.plot(x, y, 'r', linewidth=2, label=f'Gaussian fit\nμ={mu:.2f}, σ={sigma:.2f}')
```

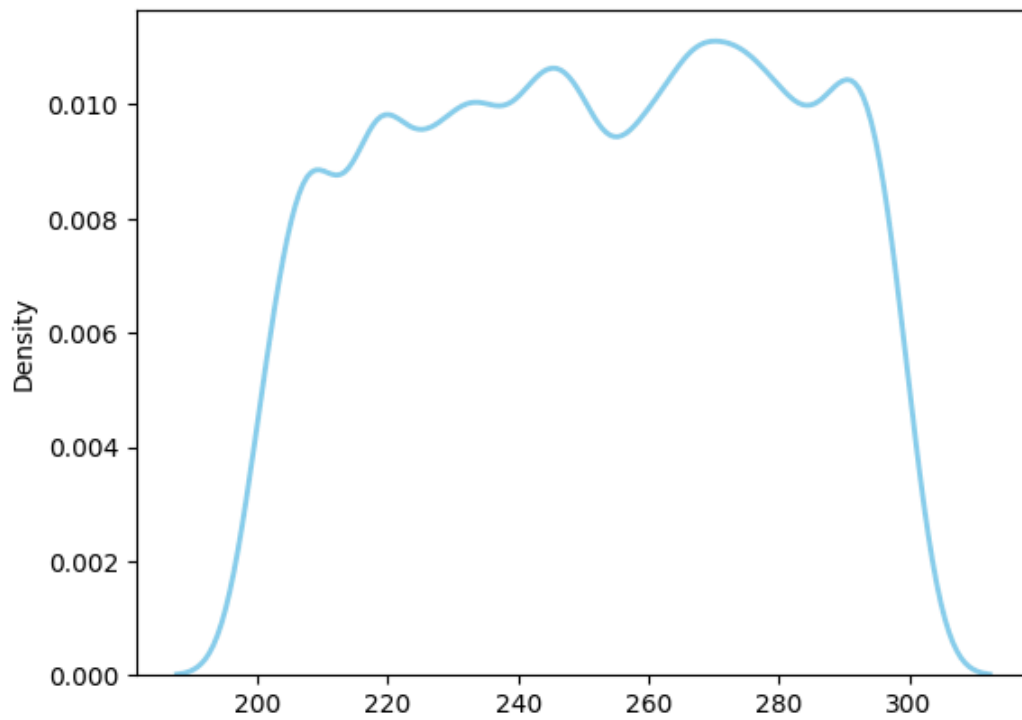
```
plt.xlabel("Value")
plt.ylabel("Probability Density")
plt.legend()
plt.show()
```



2. Frequency smoothed KDE plot : This visualization represents the data using a Kernel Density Estimation (KDE) plot, which smooths the data and provides a continuous density estimate. It shows the distribution of the data in a smooth curve, giving insights into the shape and density of the data.

```
In [17]: #In one line command
sns.kdeplot(int_list2, bw_adjust=0.5, color='skyblue', label='KDE (Smoothed Frequency)', li
```

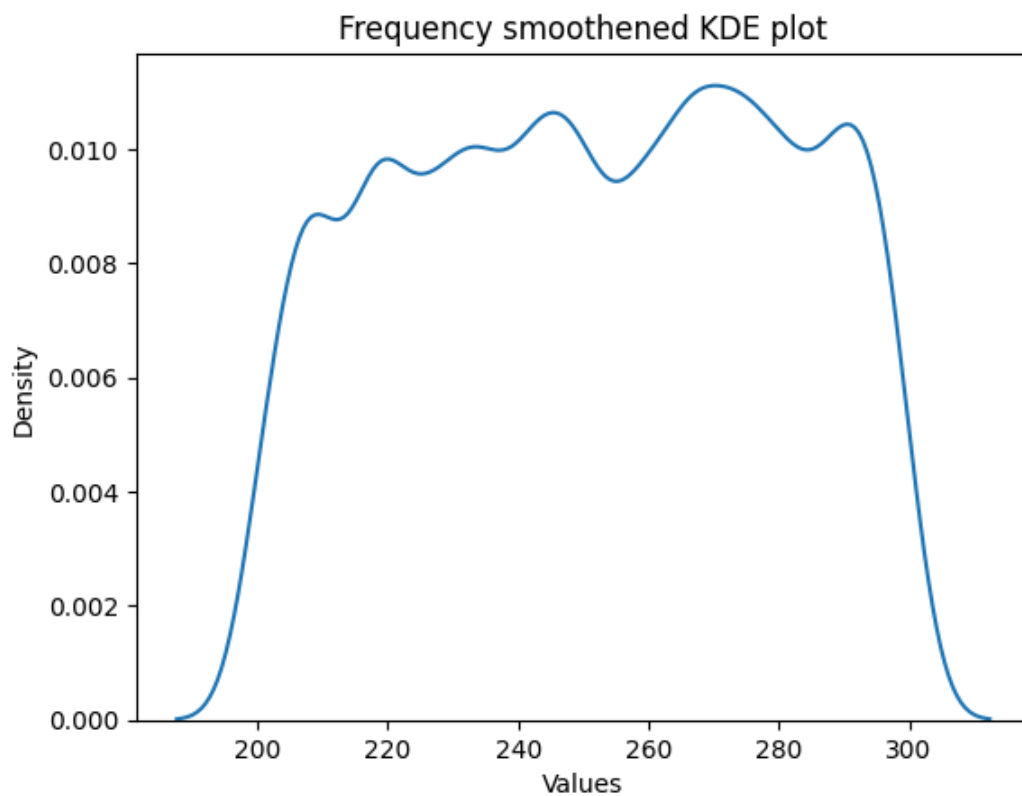
Out[17]: <Axes: ylabel='Density'>



```
In [18]: data=int_list2
values,counts=np.unique(data,return_counts=True)

#Smooth the frequency table by resampling the data:
smoothed_values = np.repeat(values, counts)

sns.kdeplot(smoothed_values,bw_adjust=0.5)
plt.xlabel("Values")
plt.ylabel("Density")
plt.title("Frequency smoothened KDE plot")
plt.show()
```



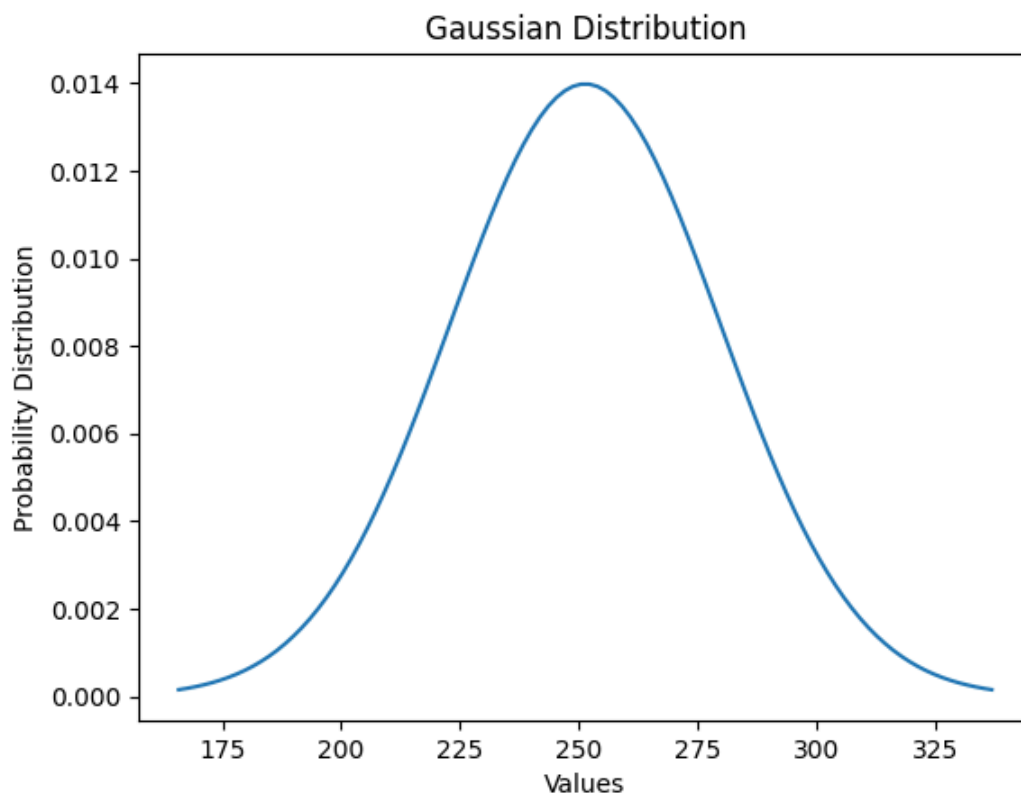
3. Gaussian distribution & smoothened KDE plot: This visualization combines the Gaussian distribution curve with the smoothened KDE plot. It allows for a comparison between the actual data distribution and the estimated distribution based on the KDE.

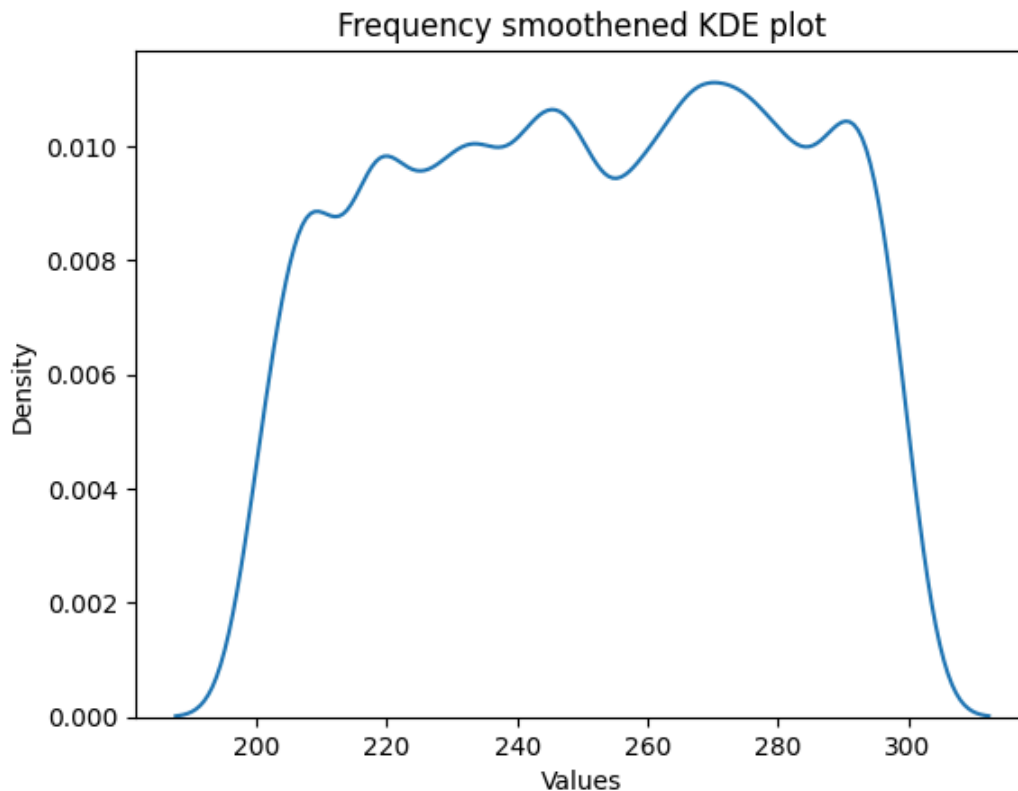
```
In [19]: #Gaussian Distribution
mu=np.mean(int_list2)
sigma=np.std(int_list2)
x=np.linspace(mu-3*sigma,mu+3*sigma,100)
y=(1/(sigma*np.sqrt(2*np.pi)))*np.exp(-0.5*((x-mu)/sigma)**2)

plt.plot(x,y)
plt.xlabel("Values")
plt.ylabel("Probability Distribution")
plt.title("Gaussian Distribution")
plt.show()

#Smoothened KDE plot
import seaborn as sns
data=int_list2
values,counts=np.unique(data,return_counts=True)
#Smooth the frequency table by resampling the data:
smoothed_values = np.repeat(values, counts)

sns.kdeplot(smoothed_values,bw_adjust=0.5)
plt.xlabel("Values")
plt.ylabel("Density")
plt.title("Frequency smoothened KDE plot")
plt.show()
```





(ii) Write a python function to calculate the range of a given list of numbers

```
In [20]: def range_(l):
          return max(l)-min(l)
          print('Range:',range_(int_list2))
```

Range: 100

(iii) Create a program to find the variance and standard deviation of list of numbers.

```
In [23]: def Variance(l,dof):
          n=len(l)
          #Find out mean
          mean=sum(l)/n
          #Deviation
          deviation=[(x-mean)**2 for x in l]
          variance=sum(deviation)/(n-dof)
          return variance
          Sample_Variance=Variance(int_list2,1)
          Population_Variance=Variance(int_list2,0)
          Sample_Std = round(np.sqrt(Sample_Variance), 2)
          Population_Std = round(np.sqrt(Population_Variance), 2)
          print("Sample Variance:",round(Sample_Variance,2))
          print("population Variance",round(Population_Variance,2))
          print("Sample Standard Deviation:", Sample_Std)
          print("Population Standard Deviation:", Population_Std)
```

Sample Variance: 816.18
 population Variance 814.54
 Sample Standard Deviation: 28.57
 Population Standard Deviation: 28.54

```
In [24]: # using Numpy
          var_p = round(np.var(int_list2, ddof=0),2) #ddof=0 for population n
          var_s = round(np.var(int_list2, ddof=1),2) #ddof=1 for sample n-1
          std_dev_p = round(np.std(int_list2, ddof=0),2)
          std_dev_s = round(np.std(int_list2, ddof=1),2)
```

```

print("Population Variance",var_p)
print("Sample Variance",var_s)
print("Population Standard Deviation",std_dev_p)
print("Sample Standard Deviation",std_dev_s)

```

Population Variance 814.54
Sample Variance 816.18
Population Standard Deviation 28.54
Sample Standard Deviation 28.57

(iv) Implement a function to compute the interquartile range (IQR) of a list of values.

```

In [25]: def IQR(l):
          q1,q3=np.percentile(l,[25,75])
          return q3-q1
          print("IQR:",IQR(int_list2))

```

IQR: 48.25

```

In [26]: def compute_iqr(values):
          q1 = np.percentile(values,25)
          q3 = np.percentile(values,75)
          iqr = q3-q1
          return iqr
          iqr_vaule = compute_iqr(int_list2)
          print("IQR", iqr_vaule)

```

IQR 48.25

(v) Build a program to calculate the coefficient of variation for a dataset.

```

In [27]: def coefficient_of_variation(data):
          mean = np.mean(data)
          std_dev = np.std(data)
          coefficient = (std_dev / mean) * 100
          return coefficient

          cv = coefficient_of_variation(int_list2).round(2)
          print("The coefficient of variation is:", cv)

```

The coefficient of variation is: 11.35

(vi) Write a python function to find the mean absolute deviation (MAD) of a list of numbers.

```

In [28]: def mean_absolute_deviation(numbers):
          mean = sum(numbers) / len(numbers)
          deviations = [abs(X - mean) for X in numbers]
          mad = sum(deviations) / len(numbers)
          return mad

          data = int_list2
          mad = mean_absolute_deviation(data).round(2)
          print("The Mean Absolute Deviation is:", mad)

```

The Mean Absolute Deviation is: 24.69

(vii) Create a program to calculate the quartile deviation of a list of values.

```

In [29]: def quartile_deviation(data):
          q1 = np.percentile(data, 25)
          q3 = np.percentile(data, 75)
          deviation = (q3 - q1) / 2
          return deviation

```

```
dataset = int_list2
qd = quartile_deviation(dataset)
print("The quartile deviation is:", qd)
```

The quartile deviation is: 24.125

(viii) Implement a function to find the range-based coefficient of dispersion for a dataset.

```
In [30]: def range_coefficient_of_dispersion(data):
        coefficient = (max(data)-min(data))/(max(data)+min(data))
        return coefficient

dataset = int_list2
rcd = range_coefficient_of_dispersion(dataset).round(2)
print("The range-based coefficient of dispersion is:", rcd)
```

The range-based coefficient of dispersion is: 0.2

3. Write a Python class representing a discrete random variable with methods to calculate its expected value and variance.

```
In [31]: class DiscreteRandomVariable:
        def __init__(self, values, probabilities):
            self.values=values
            self.probabilities=probabilities
        def expected_value(self):
            return sum(value*probability for value, probability in zip(self.values,self.probabi

        def variance(self):
            expected_value=self.expected_value()
            return sum((value-expected_value)**2*probability for value,probability in zip(self.

values = [1, 2, 3, 4]
probabilities = [0.2, 0.3, 0.4, 0.1]

rv = DiscreteRandomVariable(values, probabilities)
print("Expected Value:", round(rv.expected_value(),2))
print("Variance:", round(rv.variance(),2))
```

Expected Value: 2.4

Variance: 0.84

```
In [41]: #Another Way
class DiscreteRandomVariable:
    def __init__(self, distribution):
        self.distribution = distribution
    def expected_value(self):
        return sum(x*p for x, p in self.distribution.items())
    def variance(self):
        mean = self.expected_value()
        return sum((x-mean)**2*p for x,p in self.distribution.items())

dist = {1:0.2, 2:0.3, 3:0.4, 4:0.1}
rv = DiscreteRandomVariable(dist)

print("Expected Value",round(rv.expected_value(),2))
print("Variance", round(rv.variance(),2))
```

Expected Value 2.4

Variance 0.84

4. Implement a program to simulate the rolling of a fair six-sided die and calculate the expected value and variance of the outcomes.

```
In [47]: import random

def roll_die():
    return random.randint(1, 6)

def simulate_rolls(num_rolls):
    rolls = [roll_die() for i in range(num_rolls)]
    return rolls

def calculate_expected_value(rolls):
    return sum(rolls) / len(rolls)

def calculate_variance(rolls):
    expected_value = calculate_expected_value(rolls)
    squared_diff = [(roll - expected_value) ** 2 for roll in rolls]
    return sum(squared_diff) / len(rolls)

rolls = simulate_rolls(100)

expected_value = calculate_expected_value(rolls)
variance = calculate_variance(rolls)

print("Expected Value:", expected_value)
print("Variance:", variance)
```

Expected Value: 3.6
Variance: 2.7800000000000007

```
In [48]: #Another Way

import random
import statistics

def simulate_die_rolls(n=100):
    outcomes = [random.randint(1,6) for i in range(n)]
    return outcomes

rolls = simulate_die_rolls(100)

exp_val = statistics.mean(rolls)
var = statistics.pvariance(rolls)

print("Expected Value", exp_val)
print("Variance", var)
```

Expected Value 3.69
Variance 2.9339

5. Create a Python function to generate random sample from a given probability distribution(eg. binomial,poisson) and calculate their mean and variance.

```
In [68]: def generate_sample(distribution,size):
    if distribution == "binomial":
        sample=np.random.binomial(n=10,p=0.5,size=size)
    elif distribution == "poisson":
        sample=np.random.poisson(lam=5,size=size)
    else:
```

```

        return "Invalid distribution. Please choose either 'binomial' or 'poisson'."

    mean=np.mean(sample)
    variance=np.var(sample)
    return sample,mean, variance

samples_binomial, binomial_mean, binomial_variance = generate_sample("binomial", 1000)
print("Mean:", round(binomial_mean,2),"Variance:", round(binomial_variance,2))
samples_poisson, poisson_mean, poisson_variance = generate_sample("poisson", 1000)
print("Mean:", round(poisson_mean,2),"Variance:", round(poisson_variance,2))

```

Mean: 5.02 Variance: 2.54
Mean: 5.06 Variance: 4.86

In [70]: *#Another way*

```

def generate_samples(distribution, params, size=1000):
    if distribution == "binomial":
        samples = np.random.binomial(params["n"], params["p"], size)
    elif distribution == "poisson":
        samples = np.random.poisson(params["lam"], size)
    else:
        raise ValueError("Unsupported distribution. Use 'binomial' or 'poisson'.")

    mean = np.mean(samples)
    variance = np.var(samples)

    return samples, mean, variance

samples_binomial, mean_binomial, var_binomial = generate_samples(
    distribution="binomial",
    params={"n": 10, "p": 0.5},
    size=1000
)

samples_poisson, mean_poisson, var_poisson = generate_samples(
    distribution="poisson",
    params={"lam": 5},
    size=1000
)

print("Binomial Distribution → Mean:", mean_binomial, "Variance:", var_binomial)
print("Poisson Distribution → Mean:", mean_poisson, "Variance:", var_poisson)

```

Binomial Distribution → Mean: 5.017 Variance: 2.5687110000000004
Poisson Distribution → Mean: 5.027 Variance: 5.1862710000000005

6. Write a Python script to generate random numbers from a Gaussian (normal) distribution and compute the mean, variance, and standard deviation of the samples.

In [75]:

```

def generate_gaussian_samples(mean, std_dev, size):
    samples = np.random.normal(mean, std_dev, size)
    sample_mean = np.mean(samples).round(4)
    sample_variance = np.var(samples).round(4)
    sample_std_dev = np.std(samples).round(4)
    return sample_mean, sample_variance, sample_std_dev

mean = 0
std_dev = 1
sample_size = 1000
sample_mean, sample_variance, sample_std_dev = generate_gaussian_samples(mean, std_dev, sample_size)
print("Mean:", sample_mean)
print("Variance:", sample_variance)
print("Standard Deviation:", sample_std_dev)

```

Mean: -0.0314
Variance: 0.9933
Standard Deviation: 0.9967

7. Use seaborn library to load 'tips' dataset. Find the following from the dataset for the columns "total_bill" and "tip":

- (i) Write a Python function that calculates their skewness.
- (ii) Create a program that determines whether the columns exhibit positive skewness, negative skewness, or is approximately symmetric.
- (iii) Write a function that calculates the covariance between two columns.
- (iv) Implement a Python program that calculates the Pearson correlation coefficient between two columns.
- (v) Write a script to visualize the correlation between two specific columns in a Pandas DataFrame using scatter plots.

```
In [3]: import seaborn as sns
tips=sns.load_dataset("tips")
tips
```

```
Out[3]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

- (i) Write a python function that calculate thir skewness.

```
In [4]: def calculate_skewness():
total_bill=tips["total_bill"]
tip=tips["tip"]
total_bill_skewness=total_bill.skew().round(2)
tip_skewness=tip.skew().round(2)
print("Skewness of total_bill column:", total_bill_skewness)
print("Skewness of tip column:", tip_skewness)
calculate_skewness()
```

Skewness of total_bill column: 1.13
Skewness of tip column: 1.47

```
In [5]: #Another Way
from scipy.stats import skew
def calculate_skew(series):
    return skew(series, bias=False)

total_bill_skew = calculate_skew(tips['total_bill']).round(2)
tip_skew = calculate_skew(tips['tip']).round(2)

print("Skewness of total_bill column",total_bill_skew)
print("Skewness of tip column",tip_skew)
```

Skewness of total_bill column 1.13
 Skewness of tip column 1.47

(ii) Create a program that determines whether the columns exhibit positive skewness, negative skewness, or approximate symmetry.

```
In [22]: for col in ['total_bill', 'tip']:
s = skew(tips[col].dropna())
print(f"{col}:{'positive Skewed' if s >0.5 else 'Negative skewed' if s<- 0.5 else 'Appr
```

total_bill:positive Skewed
 tip:positive Skewed

(iii)Write a function that calculate the covariance between two columns.

```
In [26]: covariance = tips["total_bill"].cov(tips["tip"]).round(2)
print("The covariance between 'total_bill' and 'tip' is:", covariance)
```

The covariance between 'total_bill' and 'tip' is: 8.32

(iv)Implement a python program that calculate the Pearson correlation coefficient between two columns.

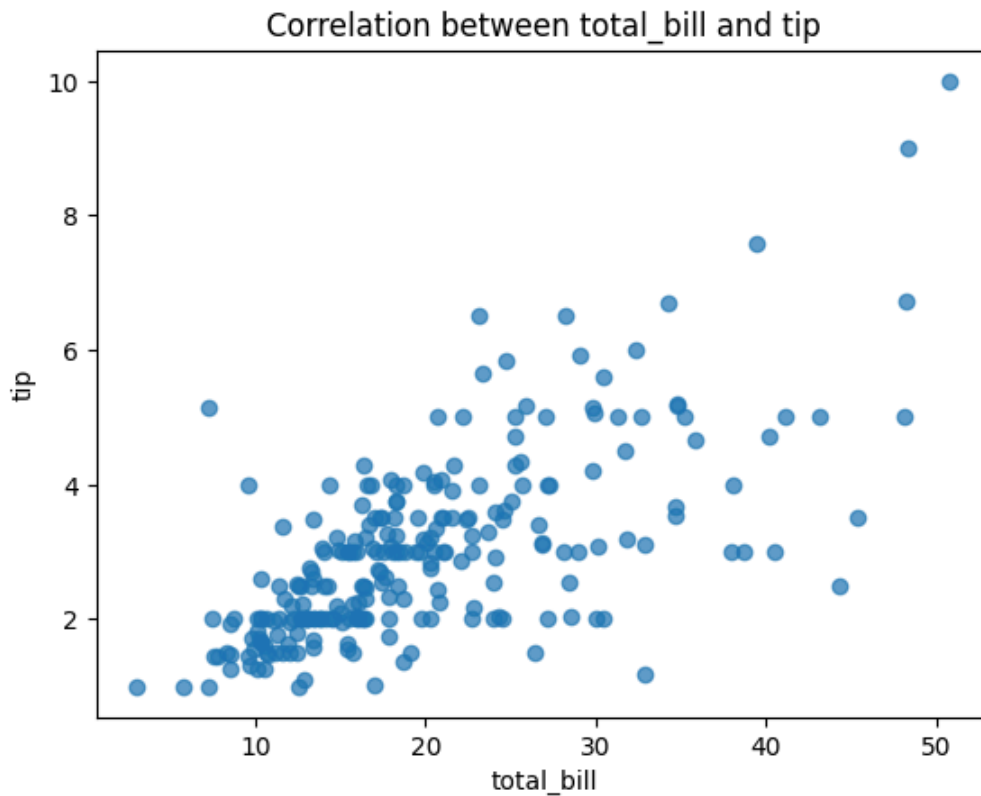
```
In [28]: correlation = tips['total_bill'].corr(tips['tip']).round(2)
print("The Pearson correlation coefficient between 'total_bill' and 'tip' is:", correlation)
```

The Pearson correlation coefficient between 'total_bill' and 'tip' is: 0.68

(v) Write a script to visualize the correlation between two specific columns in a pandas DataFrame using scatter plots.

```
In [9]: import matplotlib.pyplot as plt
x= tips["total_bill"]
y= tips["tip"]

plt.scatter(x, y, alpha =0.7)
plt.xlabel("total_bill")
plt.ylabel("tip")
plt.title("Correlation between total_bill and tip")
plt.show()
```



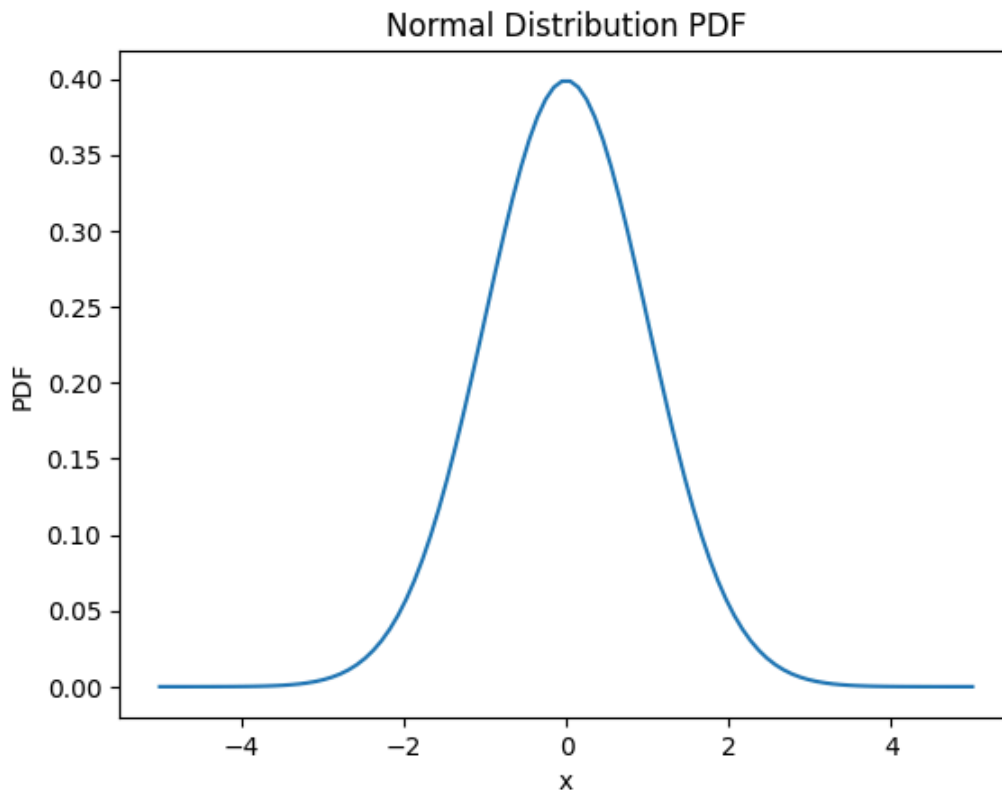
8. Write a Python function to calculate the probability density function (PDF) of a continuous random variable for a given normal distribution.

```
In [14]: from scipy.stats import norm
def normal_pdf(x, mean=0, std_dev=1):
    coeff = 1 / (std_dev * np.sqrt(2 * np.pi))
    exponent = np.exp(-0.5 * ((x - mean) / std_dev) ** 2)
    return coeff * exponent

x=np.linspace(-5,5,100)

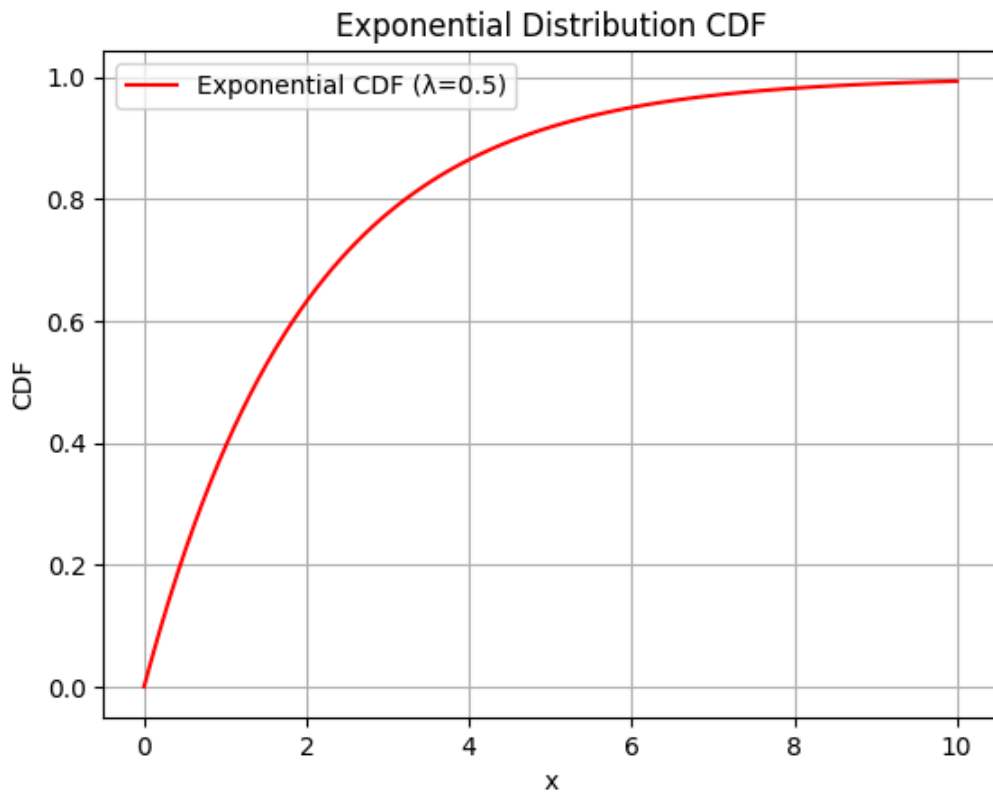
pdf=norm.pdf(x,mean,std_dev)

plt.plot(x,pdf)
plt.xlabel("x")
plt.ylabel("PDF")
plt.title("Normal Distribution PDF")
plt.show()
```

9. Create a program to calculate the cumulative distribution function (CDF) of exponential distribution.

```
In [15]: def exponential_cdf(x, lam=1.0):  
          return 1 - np.exp(-lam * x)  
  
x_values = np.linspace(0, 10, 100)  
cdf_values = exponential_cdf(x_values, lam=0.5)  
  
plt.plot(x_values, cdf_values, label="Exponential CDF ( $\lambda=0.5$ )", color="red")  
plt.xlabel("x")  
plt.ylabel("CDF")  
plt.title("Exponential Distribution CDF")  
plt.legend()  
plt.grid(True)  
plt.show()
```

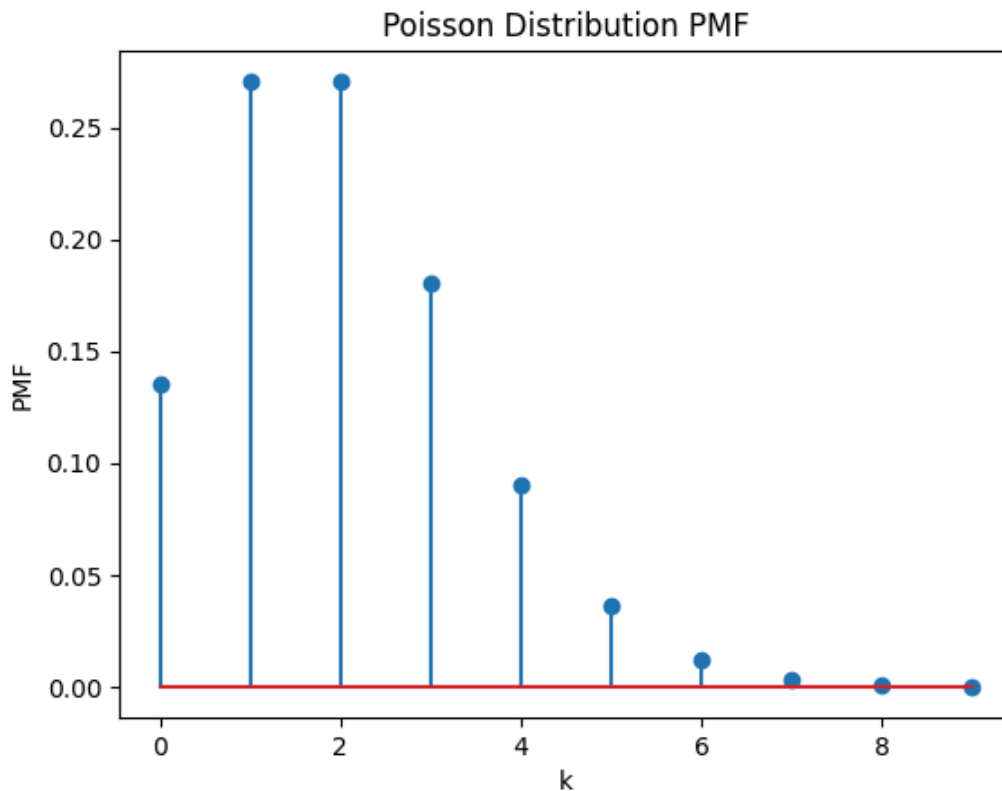


10. Write a Python function to calculate the probability mass function (PMF) of the Poisson distribution.

```
In [31]: from scipy.stats import poisson
import math
def calculate_pmf(k, lam):
    return [(math.exp(-lam) * (lam ** i)) / math.factorial(i) for i in k]

k = np.arange(0, 10) #values of k
lam = 2 #parameter lambda
pmf = calculate_pmf(k, lam)

plt.stem(k, pmf)
plt.xlabel('k')
plt.ylabel('PMF')
plt.title('Poisson Distribution PMF')
plt.show()
```



11. A company wants to test if a new website layout leads to a higher conversion rate (percentage of visitors who make a purchase). They collect data from the old and new layouts to compare. Apply the z-test to find which layout is successful.

To generate the data use following command :

```
:python
import numpy as np

#50 purchase out of 1000 visitors
old_layouts=np.array([1]*50+[0]*950)
#70 purchase out of 1000 visitors
new_layouts=np.array([1]*70+[0]*930)
```

```
In [37]: #Define the data
old_layouts = np.array([1] * 50 + [0] * 950)
new_layouts = np.array([1] * 70 + [0] * 930)
```

```
In [38]: from statsmodels.stats.proportion import proportions_ztest
#Perform the z-test
successes = np.array([np.sum(old_layouts), np.sum(new_layouts)])
nobs = np.array([len(old_layouts), len(new_layouts)])

z_score, p_value = proportions_ztest(successes, nobs)

#Interpret the results
if p_value < 0.05:
    print("The difference in conversion rates is statistically significant.")
    if z_score < 0:
        print("The new layout has a higher conversion rate.")
    else:
        print("The old layout has a higher conversion rate.")
```

```
else:
    print("There is no statistically significant difference in conversion rates.")
```

There is no statistically significant difference in conversion rates.

12. A tutoring service claims that its program improves student's exam scores. A sample of students who participated in the program was taken, and their scores before and after the program were recorded. Use the below code to generate sample of respective array of marks:

```
before_program=np.array([75,80,85,70,90,78,92,88,82,87])
after_program=np.array([80,85,90,80,92,80,95,90,85,88])
```

Use z-test to find if the claims made by the tutor are true or false.

```
In [13]: import numpy as np
from scipy.stats import norm

#Define the data
before_program = np.array([75, 80, 85, 70, 90, 78, 92, 88, 82, 87])
after_program = np.array([80, 85, 90, 80, 92, 80, 95, 90, 85, 88])

#Calculate the mean and standard deviation of the differences
differences = after_program - before_program
mean_diff = np.mean(differences)
std_diff = np.std(differences, ddof=1) / np.sqrt(len(differences))

#Perform the z-test
z_score = (mean_diff - 0) / std_diff
p_value = (1 - norm.cdf(abs(z_score)))

#Interpret the results
if p_value < 0.05:
    print("The tutoring program has a statistically significant impact on exam scores.")
    if z_score > 0:
        print("The students' scores improved after the program.")
    else:
        print("The students' scores decreased after the program.")
else:
    print("There is no statistically significant impact of the tutoring program on exam scores.")

print(f"Z-Score {z_score:.2f}")
print(f"P-value {p_value:.2e}")
```

The tutoring program has a statistically significant impact on exam scores.

The students' scores improved after the program.

Z-Score 4.59

P-value 2.18e-06

13. A pharmaceutical company wants to determine if a new drug is effective in reducing blood pressure. They conduct a study and record blood pressure measurements before and after administering the drug. Use the below code to generate a sample of respective arrays of blood pressure:

```
before_drug=np.array([145,150,140,135,155,160,152,148,130,138])
after_drug=np.array([130,140,132,128,145,148,138,136,125,130])
```

Implement z_test to find if the drug really works or not.

In [22]: `from scipy.stats import norm`

```
# Define the data
before_drug = np.array([145, 150, 140, 135, 155, 160, 152, 148, 130, 138])
after_drug = np.array([130, 140, 132, 128, 145, 148, 138, 136, 125, 130])

# Calculate the mean and standard deviation of the differences
differences = after_drug - before_drug
n = len(differences)
mean_diff = np.mean(differences)
std_diff = np.std(differences, ddof=1) / np.sqrt(n)

# Perform the z-test
z_score = (mean_diff - 0) / std_diff
p_value = 2*(1 - norm.cdf(abs(z_score)))

# Interpret the results
if p_value < 0.05:
    print("The drug has a statistically significant effect in reducing blood pressure.")
    if z_score < 0:
        print("The drug lowers blood pressure.")
    else:
        print("The drug increases blood pressure.")
else:
    print("There is no statistically significant effect of the drug in reducing blood pressure.")

print(f"Z-score {z_score:.2f}")
print(f"P-value {p_value:.2e}")
```

The drug has a statistically significant effect in reducing blood pressure.

The drug lowers blood pressure.

Z-score -10.05

P-value 0.00e+00

14.A customer service department claims that their average response time is less than 5 minutes. A sample of recent customer interaction was taken, and the response times were recorded. Implement the below code to generate the array of response times:

```
response_times=np.array([4.3,3.8,5.1,4.9,4.7,4.2,5.2,4.5,4.6,4.4])
```

Implement z_test to find the claims made by customer service department are true or false.

In [9]: `from scipy.stats import norm`

```
#Define the data
response_times = np.array([4.3, 3.8, 5.1, 4.9, 4.7, 4.2, 5.2, 4.5, 4.6, 4.4])

#Calculate the sample mean and standard deviation
sample_mean = np.mean(response_times)
sample_std = np.std(response_times, ddof=1)

#Set the null hypothesis mean and significance level
null_mean = 5
alpha = 0.05
H1 : null_mean < alpha
#Calculate the z-score
z_score = (sample_mean - null_mean) / (sample_std / np.sqrt(len(response_times)))

#Calculate the p-value
p_value = norm.cdf(z_score)

#Interpret the results
```

```

if p_value < alpha:
    print("The claims made by the customer service department are true.")
else:
    print("The claims made by the customer service department are false.")

print(f"Z-score {z_score:.2f}")
print(f"P-value {p_value:.2e}")

```

The claims made by the customer service department are true.
Z-score -3.18
P-value 7.25e-04

15.A company is testing two different website layouts to see which one leads to higher click-through rates. Write a Python function to perform an A/B test analysis, including calculating the t-statistics, degree of freedom, and p-value:

Use the following data:
layouts_a_click=[28,32,33,29,31,34,30,35,36,37]
layouts_b_clicks=[40,41,38,42,39,44,43,41,45,47]

```

In [20]: import scipy.stats as stats

def ab_test(layouts_a_click, layouts_b_clicks):
    t_stat, p_value = stats.ttest_ind(layouts_a_click, layouts_b_clicks, equal_var=True)
    dof = len(layouts_a_click) + len(layouts_b_clicks) - 2
    return t_stat, dof, p_value

layouts_a_click = [28, 32, 33, 29, 31, 34, 30, 35, 36, 37]
layouts_b_clicks = [40, 41, 38, 42, 39, 44, 43, 41, 45, 47]
mean_a= np.mean(layouts_a_click)
mean_b= np.mean(layouts_b_clicks)
t_stat, dof, p_value = ab_test(layouts_a_click, layouts_b_clicks)
if p_value < 0.05:
    print(" Significant difference between layouts (reject H0)")
else:
    print("No significant difference between layouts (fail to reject H0)")

print(f"t_stats:{t_stat:.2f}")
print(f"dof:{dof}")
print(f"p_value: {p_value:.2e}")
print(f"Mean_A: {mean_a}")
print(f"Mean_B: {mean_b}")

if mean_a>mean_b:
    print("Layout A performs better than B")
else:
    print("Layout B performs better than A")

```

Significant difference between layouts (reject H0)
t_stats:-7.30
dof: 18
p_value: 8.83e-07
Mean_A: 32.5
Mean_B: 42.0
Layout B performs better than A

16.A pharmaceutical company wants to determine if a new drug is more effective than an existing drug in reducing cholesterol levels.Create a program to analyze the clinical trial data and calculate the t-statistic and p-value for the treatment effect.Use the following data of cholesterol level:

```
existing_drug_level=[180,182,175,185,178,172,184,179,183]
new_drug_levels=[170,172,165,168,175,173,170,178,172,176]
```

```
In [31]: import scipy.stats as stats

def analyze_clinical_trial(existing_drug_levels, new_drug_levels):
    t_stat, p_value = stats.ttest_ind(existing_drug_levels, new_drug_levels, equal_var=True)
    dof_student = len(existing_drug_levels) + len(new_drug_levels) - 2
    return t_stat, p_value, dof_student

t_critical = stats.t.ppf(1 - alpha/2, dof_student)
Mean_existing = np.mean(existing_drug_levels)
Mean_new = np.mean(new_drug_levels)
existing_drug_levels = [180, 182, 175, 185, 178, 176, 172, 184, 179, 183]
new_drug_levels = [170, 172, 165, 168, 175, 173, 170, 178, 172, 176]

t_stat, p_value, dof_student = analyze_clinical_trial(existing_drug_levels, new_drug_levels)
if abs(t_stat) > t_critical:
    print("Significant difference between the two drugs (Reject H0)")
else:
    print("No significant difference between the two drugs (Fail to Reject H0)")

print(f"t_statistics: {t_stat:.2f}")
print(f"t_critical: {t_critical:.2f} ")
print(f"p_value:{p_value:.2e}")
print("Degree of freedom", dof_student)

if Mean_existing > Mean_new:
    print("Existing Drug is performs better than New Drug")
else:
    print("New Drug performs better than Existing Drug")
```

```
Significant difference between the two drugs (Reject H0)
t_statistics: 4.14
t_critical: 2.10
p_value:6.14e-04
Degree of freedom 18
Existing Drug is performs better than New Drug
```

17. A school district introduces an educational intervention program to improve math scores. Write a Python function to analyze pre- and post-intervention test scores, calculating the t-statistics and p-value to determine if the intervention has a significant impact. Use the following data of test scores:

```
pre_intervention_scores=[80,85,90,75,88,82,92,78,85,87]
post_intervention_score=[90,92,88,92,95,91,96,93,89,93]
```

```
In [45]: import scipy.stats as stats

def analyze_intervention(pre_intervention_scores, post_intervention_scores):
    t_stat, p_value = stats.ttest_rel(pre_intervention_scores, post_intervention_scores) #
    dof = len(pre_intervention_scores)-1 #degrees of freedom for paired t-test
    return t_stat, p_value, dof

t_critical = stats.t.ppf(1 - alpha/2, dof)
pre_intervention_scores = [80, 85, 90, 75, 88, 82, 92, 78, 85, 87]
post_intervention_scores = [90, 92, 88, 92, 95, 91, 96, 93, 89, 93]

Mean_pre = np.mean(pre_intervention_scores)
Mean_pro = np.mean(post_intervention_scores)

t_stat, p_value, dof = analyze_intervention(pre_intervention_scores, post_intervention_scores)
```

```

if abs(t_stat) > t_critical:
    print("Significant difference between the Pre and Post Intervention (Reject H0)")
else:
    print("No significant difference between the Pre and Post Intervention (Fail to Reject H0)")

print(f"t_statistics: {t_stat:.2f}")
print(f"t_critical: {t_critical:.2f}")
print(f"p_value:{p_value:.2e}")
print("Degrees of freedom", dof)

if Mean_pre > Mean_pro:
    print("Pre Intervention performs better than Post Intervention")
else:
    print("Post Intervention performs better than Pre Intervention")

```

```

Significant difference between the Pre and Post Intervention (Reject H0)
t_statistics: -4.43
t_critical: 2.01
p_value:1.65e-03
Degrees of freedom 9
Post Intervention performs better than Pre Intervention

```

18. An HR department wants to investigate if there's a gender-based salary gap within the company. Develop a program to analyze salary data, calculate the t-statistics, and determine if there's a statistically significant difference between the average salaries of male and female employees. Use the below code to generate synthetic data:

```

Generate synthetic salary data for male and female employees
np.random.seed(0) #For reproducibility
male_salaries=np.random.normal(loc=50000,scale=10000,size=20)
female_salaries=np.random.normal(loc=55000,scale=9000,size=20)

```

```

In [43]: import scipy.stats as stats
# Generate synthetic salary data for male and female employees

np.random.seed(0) # For reproducibility
male_salaries = np.random.normal(loc=50000, scale=10000, size=20)
female_salaries = np.random.normal(loc=55000, scale=9000, size=20)
alpha = 0.05
def analyze_salary_gap(male_salaries, female_salaries):
    t_stat, p_value = stats.ttest_ind(male_salaries, female_salaries, equal_var = True)
    dof = len(male_salaries)+len(female_salaries)-2
    return t_stat, p_value,dof

t_stat, p_value, dof = analyze_salary_gap(male_salaries, female_salaries)

if p_value < alpha:
    print("Significant salary gap found (Reject H0)")
else:
    print("No significant salary gap (Fail to Reject H0)")

print(f"t_statistics: {t_stat:.2f}")
print(f"p_value:{p_value:.2e}")
print("Degrees of freedom",dof)

```

```

No significant salary gap (Fail to Reject H0)
t_statistics: 0.06
p_value:9.52e-01
Degrees of freedom 38

```

19. A manufacturer produce two different versions of a product and wants to compare their quality score. Create a Python function to analyze quality assesment data, calculate the t-statistic, and decide

whether there's significant difference in quality between the two versions. Use the following data:

```
version1_scores=[85,88,82,89,87,84,90,88,85,86,91,83,87,84,89,86,84,88,85,86,89,90,87,88,85]
version2_scores=[80,78,83,81,79,82,76,80,78,81,77,82,80,79,82,79,80,81,79,82,79,78,80,81,82]
```

```
In [51]: import scipy.stats as stats

version1_scores = [85, 88, 82, 89, 87, 84, 90, 88, 85, 86, 91, 83, 87, 84, 89, 86, 84, 88, 85, 86, 89, 90, 87, 88, 85]
version2_scores = [80, 78, 83, 81, 79, 82, 76, 80, 78, 81, 77, 82, 80, 79, 82, 79, 80, 81, 79, 82, 79, 78, 80, 81, 82]

def compare_product_versions(version1_scores, version2_scores):
    t_stat, p_value = stats.ttest_ind(version1_scores, version2_scores)
    dof = len(version1_scores)+len(version2_scores)-2
    return t_stat, p_value, dof
Mean_V1 = np.mean(version1_scores)
Mean_V2 = np.mean(version2_scores)
t_stat, p_value, dof = compare_product_versions(version1_scores, version2_scores)
if p_value < alpha:
    print("Significant difference in product quality (Reject H0)")
else:
    print("No significant difference in product quality (Fail to Reject H0)")

print(f"t_statistics:{t_stat:.2f}")
print(f"p_value: {p_value:.2e}")
print("Degree of freedom", dof)

if Mean_V1>Mean_V2:
    print("Version 1 performs better than Version 2")
else:
    print("Version 2 performs better than Version 1")
```

```
Significant difference in product quality (Reject H0)
t_statistics:11.33
p_value: 3.68e-15
Degree of freedom 48
Version 1 performs better than Version 2
```

20.A restaurant chain collects customer satisfaction scores for two different branches. Write a program to analyze the score, calculate the t-statistic, and determine if there's statistically significant difference in customer satisfaction between the branches. Use the below data of scores:

```
branch_a_scores=[4,5,3,4,5,4,5,3,4,4,5,4,4,3,4,5,5,4,3,4,5,4,3,5,4,4,5,3,4,5,4]
branch_b_scores=[3,4,2,3,4,3,4,2,3,3,4,3,3,2,3,4,4,3,2,3,4,3,3,4,2,3,4,3]
```

```
In [52]: import scipy.stats as stats

branch_a_scores = [4, 5, 3, 4, 5, 4, 5, 3, 4, 4, 5, 4, 4, 3, 4, 5, 5, 4, 3, 4, 5, 4, 3, 5, 4, 4, 5, 3, 4, 5, 4]
branch_b_scores = [3, 4, 2, 3, 4, 3, 4, 2, 3, 3, 4, 3, 3, 2, 3, 4, 4, 3, 2, 3, 4, 3, 3, 4, 2, 3, 4, 3, 2, 4, 3]

def analyze_customer_satisfaction(branch_a_scores, branch_b_scores):
    t_stat, p_value = stats.ttest_ind(branch_a_scores, branch_b_scores)
    dof = len(branch_a_scores)+len(branch_b_scores)-2
    return t_stat, p_value, dof
Mean_a = np.mean(branch_a_scores)
Mean_b = np.mean(branch_b_scores)
t_stat, p_value, dof = analyze_customer_satisfaction(branch_a_scores, branch_b_scores)
if p_value < 0.05:
    print("There is a statistically significant difference in customer satisfaction between")
else:
```

```

    print("There is no statistically significant difference in customer satisfaction between

print(f"T-statistics {t_stat:.2f}")
print(f"P_value {p_value:.2f}")
print("Degree of freedom: ",dof)

if Mean_a>Mean_b:
    print("Branch a performs better than Branch b")
else:
    print("Branch b performs better than Branch a")

```

There is a statistically significant difference in customer satisfaction between the branches.

T-statistics 5.48

P_value 0.00

Degree of freedom: 60

Branch a performs better than Branch b

21.A political analyst wants to determine if there is a significant association between age groups and voter preferences(Candidate A or Candidate B).They collect data from a sample of 500 voters and classify them into different age groups and candidate preferences.Perform a Chi-Square test to determine if there is a significant association between age groups and voter preferences. Use the below code to generate data:

```

np.random.seed(0)
age_groups=np.random.choice(['18-30', '31-50', '51+', '51+'],size=30)
voter_prferences=np.random.choice(["Candidate A","Candidate B"],size=30)

```

In [73]: `from scipy.stats import chi2_contingency`

```

np.random.seed(0)
age_groups = np.random.choice(['18-30', '31-50', '51+'], size=30)
voter_preferences = np.random.choice(["Candidate A", "Candidate B"], size=30)

#Create a contingency table
contingency_table = np.zeros((3, 2))
for i in range(len(age_groups)):
    if age_groups[i] == '18-30':
        row = 0
    elif age_groups[i] == '31-50':
        row = 1
    else:
        row = 2
    if voter_preferences[i] == 'Candidate A':
        col = 0
    else:
        col = 1
    contingency_table[row, col] += 1

#Perform Chi-Square test
chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print(f"Chi-Square Statistic : {chi2:.4f}")
print(f"P-value : {p_value:.6f}")
print(f"Degrees of Freedom : {dof}")
print(f"Expected Freq :\n {expected}")

if p_value < 0.05:
    print("There is a significant association between age groups and voter preferences.")
else:
    print("There is no significant association between age groups and voter preferences.")

```

```

Chi-Square Statistic : 1.4402
P-value              : 0.486712
Degrees of Freedom   : 2
Expected Freq :
[[5.6      6.4      ]
 [5.13333333 5.86666667]
 [3.26666667 3.73333333]]

```

There is no significant association between age groups and voter preferences.

```

In [75]: #Another Way
from scipy.stats import chi2_contingency

# Generate synthetic data
np.random.seed(0)
age_groups = np.random.choice(['18-30', '31-50', '51+'], size=30)
voter_preferences = np.random.choice(["Candidate A", "Candidate B"], size=30)

# Create DataFrame
df = pd.DataFrame({"Age Group": age_groups, "Voter Preference": voter_preferences})

# Create contingency table
contingency_table = pd.crosstab(df["Age Group"], df["Voter Preference"])

print(f"Contingency Table :{contingency_table}")

# Perform Chi-Square test
chi2, p, dof, expected = chi2_contingency(contingency_table)

print(f"Chi-Square Statistic : {chi2:.4f}")
print(f"P-value              : {p:.6f}")
print(f"Degrees of Freedom   : {dof}")
print(f"Expected Freq: \n{expected}")

# Decision
alpha = 0.05
if p < alpha:
    print("\n Significant association between Age Group and Voter Preference (Reject H0)")
else:
    print("\n No significant association (Fail to Reject H0)")

```

```

Contingency Table :Voter Preference
Age Group
18-30      4      8
31-50      6      5
51+        4      3
Chi-Square Statistic : 1.4402
P-value              : 0.486712
Degrees of Freedom   : 2
Expected Freq:
[[5.6      6.4      ]
 [5.13333333 5.86666667]
 [3.26666667 3.73333333]]

```

No significant association (Fail to Reject H0)

22.A company conducted a customer satisfaction survey to determine if there is a significant relationship between product satisfaction levels (Satisfied, Neutral, Dissatisfied) and the region where customers are located (East,West,North,South).The survey data is summarized in a contingency table. Conduct a Chi-Square test to determine if there is a significant relationship between product satisfaction levels and customer regions.

Sample data:

```
#Sample data: Product satisfaction Levels(row) vs. Customer regions(columns)
data=np.array([[50,30,40,20],[30,40,30,50],[20,30,40,30]])
```

```
In [89]: from scipy.stats import chi2_contingency
data = np.array([[50, 30, 40, 20], [30, 40, 30, 50], [20, 30, 40, 30]])

#Perform Chi-Square test
chi2, p_value, dof, expected = chi2_contingency(data)

if p_value < 0.05:
    print("There is a significant relationship between product satisfaction levels and cust
else:
    print("There is no significant relationship between product satisfaction levels and cus

print(f"Chi-Square Statistic: {chi2:.2f}")
print("Degrees of Freedom:", dof)
print(f"P-Value: {p_value:.4f}")
print(f"Expected Frequencies: \n{expected}")
```

There is a significant relationship between product satisfaction levels and customer region s.

Chi-Square Statistic: 27.78

Degrees of Freedom: 6

P-Value: 0.0001

Expected Frequencies:

```
[[34.14634146 34.14634146 37.56097561 34.14634146]
 [36.58536585 36.58536585 40.24390244 36.58536585]
 [29.26829268 29.26829268 32.19512195 29.26829268]]
```

23.A company implemented an employee training program to improve job performance (Effective, Neutral, Ineffective). After the training, the collected data from a sample of employees and classified them based on their job performance before and after the training. Perform a Chi-Square test to determine if there is a significant difference between job performance levels before and after the training. Sample data:

```
#Sample data:Job performance Levels before(rows) and after(columns) training
data=np.array([[50,30,20],[30,40,30],[20,30,40]])
```

```
In [90]: from scipy.stats import chi2_contingency

data = np.array([[50, 30, 20], [30, 40, 30], [20, 30, 40]])

# Perform Chi-Square test
chi2, p_value, _, _ = chi2_contingency(data)

if p_value < 0.05:
    print("There is a significant difference between job performance levels before and after
else:
    print("There is no significant difference between job performance levels before and after

print("Chi-Square Statistic:", chi2)
print("Degrees of Freedom:", dof)
print("P-Value:", p_value)
print("Expected Frequencies:\n", expected)
```

There is a significant difference between job performance levels before and after the training.
 Chi-Square Statistic: 22.161728395061726
 Degrees of Freedom: 6
 P-Value: 0.00018609719479882554
 Expected Frequencies:
 [[34.14634146 34.14634146 37.56097561 34.14634146]
 [36.58536585 36.58536585 40.24390244 36.58536585]
 [29.26829268 29.26829268 32.19512195 29.26829268]]

24.A company produces three different versions of a product: Standard, Premium, and Deluxe. The company wants to determine if there is a significant difference in customer satisfaction scores among the three product versions. They conducted a survey and collected customer satisfaction scores for each version from a random sample of customers. Perform an ANOVA test to determine if there is a significant Use the following data:

#Sample data: Customer satisfaction scores for each product version
 standard_scores=[80,85,90,78,88,82,92,78,85,87]
 premium_scores=[90,92,88,92,95,91,96,93,89,93]
 deluxe_scores=[95,98,92,97,96,94,98,97,92,99]

```
In [94]: from scipy.stats import f_oneway

standard_scores = [80, 85, 90, 78, 88, 82, 92, 78, 85, 87]
premium_scores = [90, 92, 88, 92, 95, 91, 96, 93, 89, 93]
deluxe_scores = [95, 98, 92, 97, 96, 94, 98, 97, 92, 99]

#Perform ANOVA test
f_stat, p_value = f_oneway(standard_scores, premium_scores, deluxe_scores)

if p_value < 0.05:
    print("There is a significant difference in customer satisfaction scores among the three versions.")
else:
    print("There is no significant difference in customer satisfaction scores among the three versions.")

print(f"F-Statistic: {f_stat:.2f}")
print(f"P-Value: {p_value:.2e}")
```

There is a significant difference in customer satisfaction scores among the three product versions.
 F-Statistic: 27.04
 P-Value: 3.58e-07